

Relatório: Classificação de Folhas de Milho – Saudável vs. Infectada

iCev - Pós Graduação em Inteligência Artificial

Disciplina: Visão Computacional

Alunos:

- Álvaro Teixeira de Araújo
- Renan Almeida Ferreira

Dataset: Corn Leaf Infection Dataset

Notebook: `analise_corn_leaf.ipynb`

1. Dataset

1.1 Descrição

O dataset utilizado é composto por fotografias de folhas de milho (*Zea mays*) classificadas em duas categorias:

- Saudável (`Healthy corn/`)** – folhas sem sinais de doença
- Infectada (`Infected/`)** – folhas com sintomas de doenças fúngicas, como:
 - Northern Corn Leaf Blight* – *Exserohilum turcicum*
 - Common Rust* – *Puccinia sorghi*
 - Gray Leaf Spot* – *Cercospora zeae-maydis*

1.2 Estatísticas

Classe	Quantidade	Proporção
Saudável	2.000 imagens	47,3%
Infectada	2.225 imagens	52,7%
Total	4.225 imagens	100%

O dataset apresenta leve **desbalanceamento** (~5 p.p. de diferença entre as classes), o que motivou o uso de métricas robustas a desequilíbrios, como acurácia balanceada e Kappa de Cohen.

As imagens são fotografias coloridas em formato `.jpg`, capturadas em campo com variações de iluminação, ângulo e fundo.

1.3 Divisão Treino/Teste

A separação foi realizada de forma **estratificada** (proporções preservadas em ambas as partições):

Partição	Total	Saudável	Infectada
Treino	3.380	1.600	1.780
Teste	845	400	445

2. Metodologia

O pipeline de classificação foi desenvolvido em duas abordagens complementares: **modelos clássicos de ML** com extração manual de atributos e **rede neural convolucional (CNN)** com aprendizado de representações direto dos pixels.

2.1 Pré-processamento

Todas as imagens foram submetidas ao seguinte pipeline:

- 1. **Leitura** com `skimage.io.imread`
- 2. **Redimensionamento** para `128 × 128 × 3` pixels com `skimage.transform.resize`
- 3. **Normalização** para o intervalo `[0, 1]`

A padronização do tamanho é necessária para garantir compatibilidade entre modelos. O tamanho `128 × 128` foi escolhido como compromisso entre riqueza de informação visual e custo computacional.

2.2 Extração de Atributos (Modelos Clássicos)

Para os modelos clássicos, cada imagem é representada por um **vetor de atributos concatenado**, extraído com a biblioteca `scikit-image`:

Grupo de Atributos	Método	Dimensão
Histogramas de cor	<code>np.histogram</code> — 32 bins × 3 canais RGB	96
Momentos estatísticos	Média, DP, Assimetria, Curtose × 3 canais	12

Grupo de Atributos	Método	Dimensão
GLCM (textura)	<code>graycomatrix</code> + <code>graycoprops</code> – contraste, dissimilaridade, homogeneidade, energia, correlação, ASM	18
LBP	<code>local_binary_pattern</code> – histograma uniforme (radius=3, n_points=24)	26
HOG	<code>hog</code> – orientações=9, células 16×16, blocos 2×2	variável

Justificativa dos grupos:

- **Histogramas de cor:** folhas infectadas tendem a apresentar tons amarelados/marrons distintos das folhas verdes saudáveis
- **GLCM:** lesões causam padrões de textura irregulares detectáveis pela matriz de co-ocorrência
- **LBP:** captura micropadrões locais de superfície (manchas, lesões, bordas)
- **HOG:** detecta gradientes de intensidade associados a borda de lesões

2.3 Pipeline dos Modelos Clássicos

Cada classificador foi encapsulado em um `Pipeline` do scikit-learn:

StandardScaler → PCA (95% variância) → Classificador

- **StandardScaler:** normaliza cada atributo para média zero e variância unitária
- **PCA:** reduz dimensionalidade preservando 95% da variância explicada, evitando *overfitting* e acelerando o treinamento

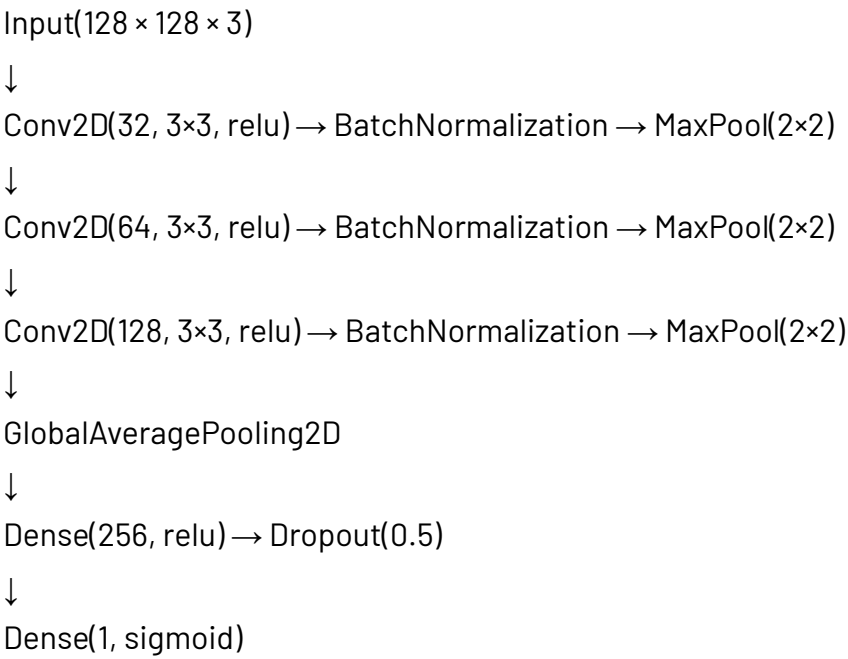
Modelos Avaliados

Modelo	Parâmetros Principais
SVM (kernel RBF)	<code>C=10</code> , <code>gamma='scale'</code> , <code>probability=True</code>
Random Forest	<code>n_estimators=200</code> , <code>max_depth=None</code>
KNN	<code>k=7</code> , <code>metric='euclidean'</code>

Antes do ajuste final, foi realizada **validação cruzada estratificada com 5 folds** para estimativa robusta do desempenho.

2.4 CNN com Keras (TensorFlow)

A rede convolucional foi construída com a API `Sequential` do Keras:



Técnicas de regularização:

- `BatchNormalization` após cada bloco convolucional: estabiliza o treinamento
- `Dropout(0.5)` na camada densa: reduz overfitting
- `GlobalAveragePooling` no lugar de `Flatten` : reduz parâmetros e melhora generalização

Configuração do treinamento:

- Otimizador: Adam (`lr=1e-3`)
- Função de perda: `binary_crossentropy`
- Augmentação de dados (`ImageDataGenerator`): flip horizontal/vertical, rotação ±20°, zoom ±15%, deslocamento ±10%
- `EarlyStopping` (paciência=10, monitorando `val_accuracy`)
- `ReduceLROnPlateau` (fator=0.5, paciência=5)

2.5 Métricas de Avaliação

Todas as abordagens foram avaliadas com o mesmo conjunto de métricas:

Métrica	Descrição
Acurácia	Proporção de predições corretas
Acurácia Balanceada	Média da sensibilidade por classe – robusta a desbalanceamento
Precisão	Dos preditos como infectados, quantos realmente são?
Recall (Sensibilidade)	Dos realmente infectados, quantos foram detectados?
F1-Score	Média harmônica entre Precisão e Recall

Métrica	Descrição
AUC-ROC	Área sob a curva ROC – discriminabilidade independente de threshold
Average Precision	Área sob a curva Precisão-Recall
Kappa de Cohen	Concordância corrigida pelo acaso

3. Resultados

3.1 Validação Cruzada (5-Fold) – Modelos Clássicos

Modelo	Acurácia CV (Média ± DP)	Folds
SVM (RBF)	0.9237 ± 0.0079	[0.9216, 0.9127, 0.9186, 0.9334, 0.9320]
Random Forest	0.8047 ± 0.0119	[0.7885, 0.7929, 0.8195, 0.8121, 0.8107]
KNN (k=7)	0.6846 ± 0.0123	[0.6716, 0.6879, 0.6701, 0.6908, 0.7027]

A validação cruzada confirma a **robustez do SVM** com baixo desvio entre folds (±0.79 p.p.), indicando boa generalização.

3.2 Desempenho no Conjunto de Teste

Modelo	Acurácia	Ac. Balanceada	Precisão	Recall	F1-Score	AUC-ROC	Kappa
CNN (Keras)	0.9787	0.9783	0.9734	0.9865	0.9799	0.9981	0.9572
SVM (RBF)	0.9373	0.9367	0.9336	0.9483	0.9409	0.9853	0.8741
Random Forest	0.8154	0.8126	0.8004	0.8652	0.8315	0.8974	0.6280
KNN (k=7)	0.6769	0.6605	0.6246	0.9685	0.7595	0.8249	0.3313

3.3 Análise por Modelo

CNN (Keras) – Melhor Desempenho Geral

A CNN obteve o **melhor resultado em todas as métricas**, com acurácia de **97,87%** e AUC-ROC de **0.9981** no conjunto de teste. O Kappa de Cohen de **0.9572** indica concordância "quase perfeita" entre predições e rótulos reais.

Resultados por classe:

Classe	Precisão	Recall	F1-Score	Suporte
Saudável	0.98	0.97	0.98	400
Infectada	0.97	0.99	0.98	445

SVM (RBF) – Melhor Modelo Clássico

O SVM com kernel RBF apresentou excelente desempenho, com acurácia de **93,73%** e AUC-ROC de **0.9853** – próximo ao da CNN, a um custo computacional muito menor no tempo de inferência. O Kappa de **0.8741** classifica o acordo como "quase perfeito".

Classe	Precisão	Recall	F1-Score	Suporte
Saudável	0.94	0.93	0.93	400
Infectada	0.93	0.95	0.94	445

A diferença de ~4 p.p. em acurácia em relação à CNN demonstra o impacto do aprendizado de representações end-to-end versus atributos manuais.

Random Forest – Desempenho Intermediário

O Random Forest atingiu **81,54% de acurácia** e AUC-ROC de **0.8974**. O Kappa de **0.6280** indica concordância "substancial". A acurácia de treino de 100% (overfitting) frente à acurácia de teste de ~81,5% sugere que o modelo não generalizou tão bem quanto o SVM, mesmo com PCA.

KNN (k=7) – Comportamento Assimétrico

O KNN apresentou a pior acurácia geral (**67,69%**), porém com **Recall muito alto (96,85%)** para a classe infectada à custa de Precisão baixa (62,46%). Isso significa que o KNN detecta quase todas as folhas infectadas, mas gera muitos falsos positivos (folhas saudáveis classificadas como infectadas).

O Kappa de **0.3313** indica apenas concordância "razoável".

4. Discussão

4.1 Por que a CNN supera os modelos clássicos?

Os modelos clássicos dependem de atributos **projetados manualmente** (GLCM, LBP, HOG, histogramas), que são representações gerais, não otimizadas para o problema específico de infecção em folhas de milho. A CNN aprende automaticamente **filtros hierárquicos** diretamente dos pixels, do simples ao complexo:

- Camadas iniciais: bordas, gradientes, texturas primitivas
- Camadas intermediárias: formas, padrões de manchas
- Camadas finais: representações discriminativas específicas da doença

4.2 Por que o SVM supera o Random Forest e KNN?

- **SVM vs. RF:** O SVM é mais eficiente com espaços de alta dimensão após PCA. A margem máxima do SVM é particularmente adequada para separar distribuições com sobreposição, como ocorre em dados de imagem com atributos ruidosos.
- **SVM vs. KNN:** O KNN sofre com a *maldição da dimensionalidade* – em espaços de alta dimensão, as distâncias entre pontos perdem significado estatístico. Mesmo com PCA, o vetor de atributos resultante é extenso.

4.3 Considerações sobre Recall vs. Precisão

No contexto de **diagnóstico fitossanitário**, o **Recall** (sensibilidade) para a classe infectada é frequentemente mais crítico que a Precisão: é preferível identificar uma planta saudável como suspeita (falso positivo) do que deixar passar uma planta infectada (falso negativo), pois isso poderia resultar na propagação da doença para a lavoura.

Nessa perspectiva:

- **CNN** oferece o melhor equilíbrio: Recall=98,65% com Precisão=97,34%
- **KNN** detecta quase todos os casos (Recall=96,85%) mas com baixa Precisão (62,46%)

4.4 Custo Computacional

Aspecto	CNN	SVM	RF	KNN
Treino	Alto (minutos/GPU)	Médio (segundos)	Baixo (segundos)	Nenhum
Inferência	Rápido	Rápido	Médio	Lento (escala com dataset)
Memória	Alta	Baixa	Média	Alta (armazena todo o conjunto)
Interpretabilidade	Baixa	Baixa	Alta	Alta

5. Conclusões

A CNN com 3 blocos convolucionais e data augmentation atingiu **97,87% de acurácia** e **AUC-ROC de 0.9981**, demonstrando alta capacidade discriminativa para o problema de classificação binária de folhas

de milho. O modelo aprende representações robustas mesmo com o leve desbalanceamento das classes (47,3% vs. 52,7%).

O **SVM com kernel RBF** se destacou entre os modelos clássicos com **93,73% de acurácia**, validando que atributos bem projetados (combinação de GLCM, LBP, HOG e histogramas de cor) são suficientes para uma classificação de alta qualidade quando combinados com um classificador robusto.

6. Referências Técnicas

Biblioteca	Versão	Uso
scikit-image	0.26.0	Leitura, pré-processamento, GLCM, LBP, HOG
scikit-learn	1.8.0	SVM, Random Forest, KNN, métricas, pipeline, PCA
TensorFlow/Keras	2.21.0	CNN, data augmentation, callbacks
NumPy	2.4.5	Operações vetorizadas
Pandas	3.0.3	Tabulação de métricas
Matplotlib	3.10.9	Visualizações
SciPy	1.17.1	Momentos estatísticos (skewness, kurtosis)